L-Int: Lexer - Handcodierte Implementierung

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom

```
;; demo
(+ 1 222 )
```

```
<LPAREN, "(">   <PLUS, "+">   <INT, "1">   <INT, "222">   <RPAREN, ")">
```

Modellierung der Token

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
```

```java
public enum TokenKind {
    PLUS, MINUS, LPAREN, RPAREN, SEMI,
    INT,  // [0-9]+
    EOF
}

public record Token(TokenKind kind, String lexeme) {}
```

Grundstruktur eines Lexers

```
public class LispyLexerLint {
    protected String source;
    protected int current;

    public List<Token> scan(String input) {
        source = input;  current = 0;
        List<Token> tokens = new ArrayList<>();

        do tokens.add(nextToken());
        while (tokens.getLast().kind() != EOF);

        return tokens;
    }

    // Arbeitsmethode: nextToken()
    // Hilfsmethoden:  isAtEnd(), peek(), advance() ...
}
```

Partielle Beispielimplementierung Lexer für L-Int

```
protected Token nextToken() {
    skipIrrelevant();

    if (!isAtEnd()) {
        char c = advance();

        if (Character.isDigit(c)) {
            String lexeme = c + readWhile(Character::isDigit);
            return Token.from(INT, lexeme);
        }
        return switch (c) {
            case '(' -> Token.from(LPAREN);
            case '+' -> Token.from(PLUS);
            default -> throw new RuntimeException("unexpected character");
        };
    }
    return Token.from(EOF);
}
```

Hilfsfunktionen im Lexer: Zeichenhandling

```
protected boolean isAtEnd() { return current >= source.length(); }

protected char peek() { return peek(0); }
protected char peek(int la) {
    if (la < 0 || current + la >= source.length()) return '\0';
    return source.charAt(current + la);
}

protected char advance() { return isAtEnd() ? '\0' : source.charAt(current++); }

protected boolean skipIrrelevant() {}  // Homework

protected String readWhile(Predicate<Character> pred) {
    StringBuilder sb = new StringBuilder();
    while (!isAtEnd() && pred.test(peek())) sb.append(advance());
    return sb.toString();
}
```

Typische Muster für die Definition von Token

1. Schlüsselwörter
   - Ein eigenes Token (RE) für jedes Schlüsselwort, oder
   - Erkennung als Name (`ID`) und nachträglich Vergleich mit Wörterbuch
2. Operatoren
   - Ein eigenes Token für jeden Operator, oder
   - Gemeinsames Token für jede Operatoren-Klasse
3. Bezeichner: Ein gemeinsames Token für alle Namen
4. Zahlen: Ein gemeinsames Token für alle numerischen Konstanten
5. String-Literale: Ein gemeinsames Token
6. Komma, Semikolon, Klammern, ...: Je ein eigenes Token
7. Regeln für Whitespace und Kommentare etc. ...

Hinweis: Längster Match

Fehler bei der lexikalischen Analyse

```
(fi (== a 42) ...)
```

=> Was tun, wenn keines der Pattern auf den Anfang des Eingabestroms passt?

Fehler bei der lexikalischen Analyse

`(fi (== a 42) ...)`

=> Was tun, wenn keines der Pattern auf den Anfang des Eingabestroms passt?

- Aufgeben ...
- "Panic Mode": Entferne so lange Zeichen, bis ein Pattern passt.
- Ein-Schritt-Transformationen:
  - Füge fehlendes Zeichen in Eingabestrom ein.
  - Entferne ein Zeichen aus Eingabestrom.
  - Vertausche ein Zeichen:
    - Ersetze ein Zeichen durch ein anderes.
    - Vertausche zwei benachbarte Zeichen.
- Fehler-Regeln: Matche typische Typos (Fehler-Token)

Wrap-up

- Lexer zerlegt den Quelltext von links nach rechts in Token
  - Token: hat Tokenart und Lexem
  - Reguläre Ausdrücke beschreiben die Formen der Token
  - Whitespace und Kommentare überspringen
- Implementierungsansatz: Zeichenweise Iteration durch die Eingabe
  - Entscheidung für ein Token am aktuellen Zeichen
  - Ggf. Lookahead notwendig, z.B. bei "==" vs. "="
  - Bevorzugung des längsten Matches
  - Ende der Eingabe: Künstliches `EOF`-Token



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *83e910a 2026-11-04 lint: lexer: add new screencast*